

Reto AI-First · Fase 1

Diseña y ejecuta una estrategia de pruebas sobre gestor-inventario — sin escribir scripts manualmente

Track QA · Preparado por Diego Trujillo · diego.trujillo@jikkosoft.com · Jikkosoft

1. Por qué hacemos esto

Jikkosoft está dando un paso hacia un modelo de trabajo AI-first: el rol del QA Engineer deja de ser ejecutar casos manualmente y pasa a ser diseñar estrategias, orquestar pruebas y analizar evidencia con asistencia de IA.

Este reto es tu espacio para experimentar ese cambio de chip mental en un entorno seguro, con acompañamiento y una ruta de aprendizaje. No se trata de saberlo todo desde el día uno: se trata de adaptarte, investigar y construir.

2. El reto en una frase

En 6 días hábiles, diseña y ejecuta una estrategia de pruebas completa sobre gestor-inventario — la app incluida en este repositorio — sin escribir scripts a mano: todo se genera y se itera a través de Hermes como agente principal y uno o varios LLMs de su preferencia para codificar o implementar (ej: Claude Sonnet, Haiku, Opus, Codex, DeepSeek, Kimi K2, etc.). El objetivo es probar la app, no modificarla.

3. La app que debes probar — gestor-inventario

La aplicación está en `3-challenge/gestor-inventario/`. Es una app full-stack mínima y autocontenida: FastAPI + SQLite + frontend estático. No debes modificar su código — solo probarlo.

Método	Ruta	Descripción
GET	/api/health	Estado del servicio
GET	/api/suppliers	Lista de proveedores activos
GET	/api/products	Lista de productos activos

GET	/api/products/{id}	Detalle — 404 si no existe
POST	/api/products	Crear producto (201 / 409 SKU duplicado)
POST	/api/stock/movement	Movimiento IN o OUT (201 / 503 si ALERTS_FAIL)
GET	/api/stock/alerts	Productos con stock <= minimo (503 si ALERTS_FAIL)
GET	/api/movements	Historial de movimientos

Para ejecutar la app localmente:

```
cd 3-challenge/gestor-inventario
pip install -r requirements.txt && uvicorn app:app --port 8000
# Con Docker: docker compose up --build
# Fallo de integracion: ALERTS_FAIL=1 uvicorn app:app --port 8000
# DB aislada para pruebas: DB_PATH=test.db uvicorn app:app --port 8001
```

Todos los montos son en centavos (integer). La app siembra 3 proveedores y 6 productos al primer arranque.

4. Alcance de cobertura requerido

Tu estrategia debe cubrir estas dimensiones obligatoriamente:

Dimension	Que validar
Contrato de API	HTTP status codes, campos requeridos en respuesta, shapes de error
Funcional	Happy path, edge cases y casos negativos para cada endpoint
Integridad de datos	Stock se actualiza correctamente tras movimientos IN/OUT; valores match aritmetica
Fallo de integracion	ALERTS_FAIL=1 → 503 en /api/stock/alerts y POST /api/stock/movement (OUT)
UI / E2E	Flujo de registro de movimientos y consulta de alertas desde el frontend

5. Reglas del juego

1. Cero scripts manuales. No escribes codigo de pruebas tu; lo genera la IA. Tu trabajo es disenar la estrategia, dirigir la generacion, revisar e iterar.
2. Hermes como agente principal y uno o varios LLMs de su preferencia para codificar o implementar (ej: Claude Sonnet, Haiku, Opus, Codex, DeepSeek, Kimi K2, etc.) — obligatorio e innegociable. Toda interaccion con modelos pasa a traves de Hermes.
3. Tu repo, tu casa. Sube el codigo a GitHub publico o GitLab, donde prefieras.

4. No modificar la app bajo prueba. Cualquier cambio al código de gestor-inventario invalida los hallazgos.
5. Provee tu acceso a modelos. Necesitaras un proveedor LLM conectado a Hermes. Ver punto 8 sobre créditos.

6. Que debes entregar

Entregable	Descripcion
SOUL.md	Log del proceso: scope, tooling, decisiones de estrategia, hallazgos, blockers
Plan de pruebas	Estrategia, alcance, riesgos, criterios de aceptacion
Casos de prueba	Happy path + edge + negativos, documentados (markdown o tabla)
Suite API	Contrato + validacion de errores — generada con Hermes + LLM, ejecutable con pytest
Suite E2E (deseable)	Playwright o equivalente contra localhost:8000 — generada con IA si el tiempo lo permite
Reporte de defectos	Hallazgos con severidad, pasos de reproduccion, evidencia

Plantilla del entregable SOUL.md

- Proyecto: que app probaste y cual fue tu estrategia de cobertura
- Stack de pruebas: herramientas, frameworks y como se conectan
- Como usaste Hermes y los LLMs: prompts que mejor funcionaron, iteraciones de generacion
- Decisiones y trade-offs: que elegiste cubrir primero y por que
- Hallazgos: defectos encontrados con severidad y evidencia
- Bloqueos y como los resolviste
- Enlace al repositorio

7. Cronograma (tentativo)

Día	Fecha	Foco
Apertura	Vie 26 jun	Lanzamiento del reto, exploracion de la app, diseno de estrategia
Días 1-7	Lun-Dom 30 jun - 6 jul	Generacion de suites + ejecucion + reporte de defectos
Evaluacion	Mar 7 jul	Demos y revision de hallazgos por grupos

Punto de control diario: un reporte breve (3 lineas) de avance + bloqueos. Sirve para acompañarte, no para vigilarte.

8. Sobre credits y costos — lee lo

Los modelos cuestan y las capas gratuitas se agotan. Prever esto es parte del reto.

Si ves que te vas a quedar corto de credits o necesitas acceso a un modelo corporativo, levanta la mano a tiempo (no el ultimo dia). Anticiparte y comunicar a tiempo es justamente una de las capacidades que estamos observando.

9. Ruta de aprendizaje sugerida

No estas solo. Te sugerimos recursos para arrancar (no son obligatorios ni exhaustivos — exploralos y trae los tuyos).

Testing con IA — frameworks recomendados

- [Playwright — documentacion oficial](#)
- [pytest + httpx — testing de APIs async](#)

Escritura de specs y prompts efectivos

- [Guia de prompt engineering \(Claude Docs\)](#)

Hermes (obligatorio)

- [Hermes Agent \(Nous Research\) — repositorio y documentacion oficial](#)

10. Como te vamos a evaluar (vision general)

No buscamos la suite mas grande, buscamos criterio y profundidad. Valoramos especialmente:

- La calidad de tu SOUL.md y la trazabilidad de tu proceso
- Tu autonomia: como investigaste y resolviste bloqueos por tu cuenta
- La profundidad de cobertura: no solo happy path — edge cases, negativos, integridad de datos
- Los defectos encontrados y la calidad de su documentacion (severidad, repro, evidencia)
- Las suites son ejecutables: Playwright y pytest corren contra localhost:8000 sin modificacion manual
- Tu prevision y comunicacion a tiempo

Dudas durante el reto: canarizalas a Diego Trujillo — diego.trujillo@jikkosoft.com o Google Chat.